6. Gradient Boosting, XGBoost, and SHAP Values

Overview

After reading this chapter, you will be able to describe the concept of gradient boosting, the fundamental idea underlying the XGBoost package. You will then train XGBoost models on synthetic data, while learning about early stopping as well as several XGBoost hyperparameters along the way. In addition to using a similar method to grow trees as we have previously (by setting `max_depth`*), you'll also discover a new way of growing trees that is offered by XGBoost: loss-guided tree growing. After learning about XGBoost, you'll then be introduced to a new and powerful way of explaining model predictions, called* **SHAP** *(***SHapley Additive exPlanations***). You will see how SHAP values can be used to provide individualized explanations for model predictions from any dataset, not just the training data, and also understand the additive property of SHAP values.*

Introduction

As we saw in the previous chapter, decision trees and ensemble models based on them provide powerful methods for creating machine learning models. While random forests have been around for decades, recent work on a different kind of tree ensemble, gradient boosted trees, has resulted in state-of-the-art models that have come to dominate the landscape of predictive modeling with tabular data, or data that is organized into a structured table, similar to the case study data.

The two main packages used by machine learning data scientists today to create the most accurate predictive models with tabular data are XGBoost and LightGBM. In this chapter, we'll become familiar with XGBoost using a synthetic dataset, and then apply it to the case study data in the activity.

Note

Perhaps some of the best motivation for using XGBoost comes from the paper describing this machine learning system, in the context of Kaggle, a popular online forum for machine learning competitions:

"Among the 29 challenge-winning solutions published on Kaggle's blog during 2015, 17 solutions used XGBoost. Among these solutions, eight solely used XGBoost to train the model, while most others combined XGBoost with neural nets in ensembles. For comparison, the second most popular method, deep neural nets, was used in 11 solutions " (Chen and Guestrin, 2016, **https://dl.acm.org/doi/abs/10.1145/2939672.2939785***).*

As we'll see, XGBoost ties together a few of the different ideas we've discussed so far, including decision trees and ensemble modeling as well as gradient descent.

In addition to more performant models, recent machine learning research has yielded more detailed ways to explain the predictions of models. Rather than relying on interpretations that only represent the model training set in aggregate, such as logistic regression coefficients or the feature importances of a random forest, a new package called SHAP allows us to interpret model predictions individually, and for any dataset we want, such as validation or test data. This can be very helpful in enabling us, as data scientists, as well as our business partners, to understand the workings of a model at a granular level, even for new data.

Gradient Boosting and XGBoost

What Is Boosting?

Boosting is a procedure for creating ensembles of many machine learning models, or **estimators**, similar to the bagging concept that underlies the random forest model. Like bagging, while boosting can be used with any kind of machine learning model, it is commonly used to build ensembles of decision trees. A key difference from bagging is that in boosting, each new estimator added to the ensemble depends on all the estimators added before it. Because the boosting procedure proceeds in sequential stages, and the predictions of ensemble members are added up to calculate the overall ensemble prediction, it is also called **stagewise additive modeling**. The difference between bagging and boosting can be visualized as in *Figure 6.1*:

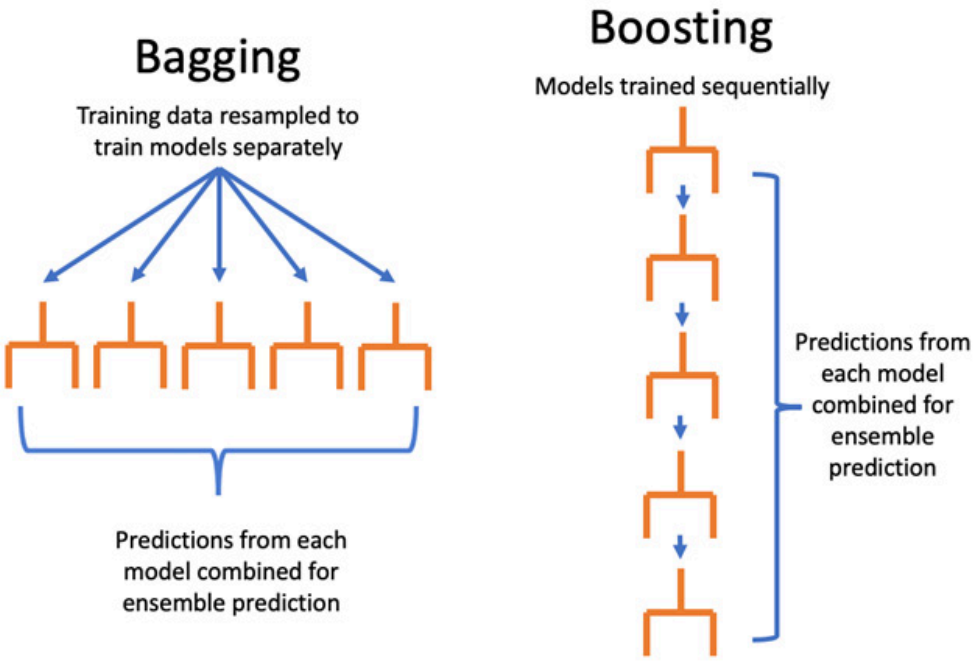

Figure 6.1: Bagging versus boosting

While bagging trains many estimators using different random samples of the training data, boosting trains new estimators using information about which samples were incorrectly classified by the previous estimators in the ensemble. By focusing new estimators on these samples,

the goal is that the overall ensemble will have better performance across the whole training dataset. **AdaBoost**, a precursor to **XGBoost**, accomplished this goal by giving more weight to incorrectly classified samples as new estimators in the ensemble are trained.

Gradient Boosting and XGBoost

XGBoost is a modeling procedure and Python package that is one of the most popular machine learning methods in use today, due to its superior performance in many domains, from business to the natural sciences. XGBoost has also proven to be one of the most successful tools in machine learning competitions. We will not discuss all the details of how XGBoost is implemented, but rather get a high-level idea of how it works and look at some of the most important hyperparameters. For further details, the interested reader should refer to the publication *XGBoost: A Scalable Tree Boosting System*, by Tianqi Chen and Carlos Guestrin (**https://dl.acm.org/doi/abs/10.1145/2939672.2939785**).

The XGBoost implementation of the gradient boosting procedure is a stagewise additive model similar to AdaBoost. However, instead of directly giving more weight to misclassified samples during model training, XGBoost uses a procedure similar in nature to gradient descent. Recall from *Chapter 4, The Bias Variance Trade-off*, that optimization with gradient descent uses information about the derivative of a **loss function** (another name for the cost function) to update the estimated coefficients when training a logistic regression model. The derivative of the loss function contains information about which direction and how much to adjust the coefficient estimates at each iteration of the procedure, so as to reduce the level of error in the predictions.

XGBoost applies the gradient descent idea to stagewise additive modeling, using information about the gradient (another word for derivative) of a loss function to train new decision trees to add to the ensemble. In fact, XGBoost takes things a step further than gradient descent

as described in *Chapter 4*, *The Bias-Variance Trade-Off*, and uses information about both the first and second derivatives. The approach of training decision trees using error gradients is an alternative to the node impurity idea introduced in *Chapter 5*, *Decision Trees and Random Forests*. Conceptually, XGBoost trains new trees with the goal of moving the ensemble prediction in the direction of decreasing error. How big a step to take in that direction is controlled by the **`learning_rate`** hyperparameter, analogous to **`learning_rate`** in *Exercise 4.01 Using Gradient Descent to Minimize a Cost Function*, from *Chapter 4*, *The Bias Variance Trade-off*.

At this point, we should have enough knowledge about how XGBoost works to start getting our hands dirty and using it. To illustrate XGBoost, we'll create a synthetic dataset for binary classification, with scikit-learn's **`make_classification`** function. This dataset will have 5,000 samples and 40 features. The rest of the options here control how challenging a classification task this will be, and you should consult the scikit-learn documentation to better understand them. Of particular interest is the fact that we'll have multiple clusters (**`n_clusters_per_class`**), meaning there will be several regions of points in multidimensional feature space that belong to a certain class, similar to the cluster shown in the last chapter in *Figure 5.3*. A tree-based model should be able to identify these clusters. Also, we are specifying that there are only 3 informative features out of a total of 40 (**`n_informative`**), as well as 2 redundant features (**`n_redundant`**) that will contain the same information as the informative ones. So, all told, only 5 out of the 40 features should be useful in making predictions, and of those, all the information is encoded in 3 of them.

If you want to follow along with the examples in this chapter on your computer, please refer to the Jupyter notebook at **https://packt.link/L5oS7**:

```
from sklearn.datasets import make_classification
```

```
X, y = make_classification(n_samples=5000,
n_features=40,\
                           n_informative=3,
n_redundant=2,\
                           n_repeated=0, n_classes=2,\
                           n_clusters_per_class=3,\
                           weights=None, flip_y=0.05,\
                           class_sep=0.1,
hypercube=True,\
                           shift=0.0,scale=1.0,
shuffle=True,\
                           random_state=2)
```

Note that the class fraction of the response variable **y** is about 50%:

```
y.mean()
```

This should output the following:

```
0.4986
```

Instead of using cross-validation, in this chapter, we will split this synthetic dataset just once into a training and validation set. However, the concepts we introduce here could be extended to the cross-validation scenario. We'll split this synthetic data into 80% for training and 20% for validation. In a real-world data problem, we would also want to have a test set reserved for later use in evaluating the final model, but we'll forego this here:

```
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = \
train_test_split(X, y, test_size=0.2, random_state=24)
```

Now that we've prepared the data for modeling, we need to instantiate an object of the **XGBClassifier** class. Note that we will now be using the XGBoost package, and not scikit-learn, to develop a predictive model. However, XGBoost has an API (application programming interface) that was designed to be similar to that of scikit-learn, so using this class should be intuitive. The **XGBClassifier** class can be used to

create a model object with **`fit`** and **`predict`** methods and other familiar functionality, and we can specify model hyperparameters when instantiating the class. We'll specify just a few hyperparameters here, which we've already discussed: **`n_estimators`** is the number of boosting rounds to use for the model (in other words, the number of stages for the stagewise additive modeling procedure), **`objective`** is the loss function that will be used to calculate gradients, and **`learning_rate`** controls how much each new estimator adds to the ensemble, or, in essence, how far of a step to take to decrease prediction error. The remaining hyperparameters are related to how much output we want to see during model training (**`verbosity`**) and the soon-to-be-deprecated **`label_encoder`** option, which XGBoost developers recommend setting to **`False`**:

```
xgb_model_1 = xgb.XGBClassifier(n_estimators=1000,\
                                verbosity=1,\
                                use_label_encoder=False
,\
                                objective='binary:logis
tic',\
                                learning_rate=0.3)
```

The hyperparameter values we've indicated specify that:

- We will have 1,000 estimators, or boosting rounds. We'll discuss in more detail shortly how many rounds are needed; the default value is 100.
- We are familiar with the objective function (also known as the cost function) for binary logistic regression from *Chapter 4*, *The Bias-Variance Trade-Off*. XGBoost also offers a wide variety of objective functions for a range of tasks, including classification and regression.
- The learning rate is set to **`0.3`**, which is the default. Different values can be explored via hyperparameter search procedures, which we'll demonstrate.

  Note

It is recommended to install XGBoost and SHAP using an Anaconda environment as demonstrated in the Preface. If you install different versions than those indicated, your results may be different than shown here.

Now that we have a model object and some training data, we are ready to fit the model. This looks similar to how it did in scikit-learn:

```
%%time
xgb_model_1.fit(X_train, y_train,\
                eval_metric="auc",\
                verbose=True)
```

Here, we are tracking how long the fitting procedure takes using the **`%%time`** "cell magic" in the Jupyter notebook. We need to supply the features **`X_train`** features and the response variable **`y_train`** of the training data. We also supply **`eval_metric`** and set the verbosity, which we'll explain shortly. Executing this cell should give output similar to this:

```
CPU times: user 52.5 s, sys: 986 ms, total: 53.4 s
Wall time: 17.5 s
Out[7]:
XGBClassifier(base_score=0.5, booster='gbtree',\
              colsample_bylevel=1, colsample_bynode=1,\
              colsample_bytree=1, gamma=0, gpu_id=-1,\
              importance_type='gain',interaction_constr
aints='',\
              learning_rate=0.3, max_delta_step=0,
max_depth=6,\
              min_child_weight=1, missing=nan,\
              monotone_constraints='()',
n_estimators=1000,\
              n_jobs=4, num_parallel_tree=1,
random_state=0,\
```

```
                reg_alpha=0, reg_lambda=1,
  scale_pos_weight=1,\
                subsample=1, tree_method='exact',\
                use_label_encoder=False,
  validate_parameters=1,\
                verbosity=1)
```

The output tells us that this cell took 17.5 seconds to execute, called the "wall time," or the elapsed time on a clock that might be on your wall. The CPU times are longer than this because XGBoost efficiently uses multiple processors simultaneously. XGBoost also prints out all the hyperparameters, including the ones we set and the others that were left at their defaults.

Now, to examine the performance of this fitted model, we'll evaluate the area under the ROC curve on the validation set. First, we need to obtain the predicted probabilities:

```
  val_set_pred_proba = xgb_model_1.predict_proba(X_val)
  [:,1]
  from sklearn.metrics import roc_auc_score
  roc_auc_score(y_val, val_set_pred_proba)
```

The output of this cell should be as follows:

```
  0.7773798710782294
```

This indicates an ROC AUC of about 0.78. This will be our model performance baseline, using nearly default options for XGBoost.

XGBoost Hyperparameters

Early Stopping

When training ensembles of decision trees with XGBoost, there are many options available for reducing overfitting and leveraging the bias-variance trade-off. **Early stopping** is a simple one of these and

can help provide an automated answer to the question "How many boosting rounds are needed?". It's important to note that early stopping relies on having a separate validation set of data, aside from the training set. However, this validation set will actually be used during the model training process, so it does not qualify as "unseen" data that was held out from model training, similar to how we used validation sets in cross-validation to select model hyperparameters in *Chapter 4, The Bias-Variance Trade-Off*.

When XGBoost is training successive decision trees to reduce error on the training set, it's possible that adding more and more trees to the ensemble will provide increasingly better fits to the training data, but start to cause lower performance on held-out data. To avoid this, we can use a validation set, also called an evaluation set or **`eval_set`** by XGBoost. The evaluation set will be supplied as a list of tuples of features and their corresponding response variables. Whichever tuple comes last in this list will be the one that is used for early stopping. We want this to be the validation set since the training data will be used to fit the model and can't provide an estimate of out-of-sample generalization:

```
  eval_set = [(X_train, y_train), (X_val, y_val)]
```

Now we can fit the model again, but this time we supply the **`eval_set`** keyword argument with the evaluation set we just created. At this point, the **`eval_metric`** of **`auc`** becomes important. This means that after each boosting round, before training another decision tree, the area under the ROC curve will be evaluated on all the datasets supplied with **`eval_set`**. Since we'll indicate **`verbosity=True`**, we'll get output printed below the cell with the ROC AUC for both the training set and the validation set. This provides a nice live look at how model performance changes on the training and validation data as more boosting rounds are trained.

Since, in predictive modeling, we're primarily interested in how a model performs on new and unseen data, we would like to stop train-

ing additional boosting rounds when it becomes clear that they are not improving model performance on out-of-sample data. The **`early_stopping_rounds=30`** argument indicates that once 30 boosting rounds have been completed without any additional improvement in the ROC AUC on the validation set, XGBoost should stop model training. Once model training is complete, the final fitted model will only have as many ensemble members as needed to get the highest model performance on the validation set. This means that the last 30 members of the ensemble will be discarded, since they didn't provide any increase in validation set performance. Let's now fit this model and watch the progress:

```
%%time
xgb_model_1.fit(X_train, y_train, eval_set=eval_set,\
                eval_metric='auc',\
                verbose=True, early_stopping_rounds=30)
```

The output should look something like this:

```
[0] validation_0-auc:0.80412 validation_1-auc:0.75223
[1] validation_0-auc:0.84422 validation_1-auc:0.79207
[2] validation_0-auc:0.85920 validation_1-auc:0.79278
[3] validation_0-auc:0.86616 validation_1-auc:0.79517
[4] validation_0-auc:0.88261 validation_1-auc:0.79659
[5] validation_0-auc:0.88605 validation_1-auc:0.80061
[6] validation_0-auc:0.89226 validation_1-auc:0.80224
[7] validation_0-auc:0.89826 validation_1-auc:0.80305
[8] validation_0-auc:0.90559 validation_1-auc:0.80095
[9] validation_0-auc:0.91954 validation_1-auc:0.79685
[10] validation_0-auc:0.92113 validation_1-auc:0.79608
…
[33] validation_0-auc:0.99169 validation_1-auc:0.78323
[34] validation_0-auc:0.99278 validation_1-auc:0.78261
[35] validation_0-auc:0.99329 validation_1-auc:0.78139
[36] validation_0-auc:0.99344 validation_1-auc:0.77994
CPU times: user 2.65 s, sys: 136 ms, total: 2.78 s
```

```
  Wall time: 2.36 s
  …
```

Notice that this took much less time than the previous fit. This is because, due to early stopping, we only trained 37 rounds of boosting (notice boosting rounds are zero indexed). This means that the boosting procedure only needed 8 rounds to achieve the best validation score, as opposed to the 1,000 we tried previously! You can access the number of boosting rounds needed to reach the optimal validation set score, as well as that score, with the **`booster`** attribute of the model object. This attribute presents a lower-level interface to the model than the scikit-learn API we have been using:

```
  xgb_model_1.get_booster().attributes()
```

The output should look like this, confirming the number of iterations and best validation score:

```
  {'best_iteration': '7', 'best_score': '0.80305'}
```

From the training procedure, we can also see the ROC AUC after each round for both the training data, **`validation_0-auc`**, and the validation data, **`validation_1-auc`**, which provide insights into overfitting as the boosting procedure progresses. Here we can see that the validation score increased up to round 8, after which it started to decrease, indicating that further boosting would likely produce an undesirably overfitted model. However, the training score continued to increase up to the point the procedure was terminated, showing how powerfully XGBoost is able to fit the training data.

We can further confirm that the fitted model object only represents seven rounds of boosting, and check validation set performance, by manually calculating the ROC AUC as we did previously:

```
  val_set_pred_proba_2 = xgb_model_1.predict_proba(X_val)
  [:,1]
  roc_auc_score(y_val, val_set_pred_proba_2)
```

This should output the following:

```
0.8030501882609966
```

This matches the highest validation score achieved after seven rounds of boosting. So, with one simple tweak to the model training procedure, by using a validation set and early stopping, we were able to improve model performance on the validation set from about 0.78 to 0.80, a substantial increase. This shows the importance of early stopping in boosting.

One natural question to ask here is "How did we know that 30 rounds for early stopping would be enough?". You can experiment with this number, as with any hyperparameter, and different values may be appropriate for different datasets. You can look to see how the validation score changes with each boosting round to get an idea for this. Sometimes, the validation score can increase and decrease in a jumpy way from round to round, so it's a good idea to have enough rounds to make sure you've found the maximum, and boosted through any temporary decreases.

Tuning the Learning Rate

The learning rate is also referred to as **eta** in the XGBoost documentation, as well as **step size shrinkage**. This hyperparameter controls how much of a contribution each new estimator will make to the ensemble prediction. If you increase the learning rate, you may reach the optimal model, defined as having the highest performance on the validation set, faster. However, there is the danger that setting it too high will result in boosting steps that are too large. In this case, the gradient boosting procedure may not converge on the optimal model, due to similar issues to those discussed in *Exercise 4.01, Using Gradient Descent to Minimize a Cost Function,* from *Chapter 4, The Bias Variance Trade-off*, regarding large learning rates in gradient descent. Let's explore how the learning rate affects model performance on our synthetic data.

The learning rate is a number between zero and 1 (inclusive of endpoints, although a learning rate of zero is not useful). We make an array of 25 evenly spaced numbers between 0.01 and 1 for the learning rates we'll test:

```
learning_rates = np.linspace(start=0.01, stop=1,
num=25)
```

Now we set up a **`for`** loop to train a model for each learning rate and save the validation scores in an array. We'll also track the number of boosting rounds that it takes to reach the best iteration. The next several code blocks should be run together as one cell in a Jupyter notebook. We start by measuring how long this will take, creating empty lists to store results, and opening the **`for`** loop:

```
%%time
val_aucs = []
best_iters = []
for learning_rate in learning_rates:
```

At each loop iteration, the **`learning_rate`** variable will hold successive elements of the **`learning_rates`** array. Once inside the loop, the first step is to update the hyperparameters of the model object with the new learning rate. This is accomplished using the **`set_params`** method, which we supply with a double asterisk **`**`** and a dictionary mapping hyperparameter names to values. The **`**`** function call syntax in Python allows us to supply an arbitrary number of keyword arguments, also called **kwargs**, as a dictionary. In this case, we are only changing one keyword argument, so the dictionary only has one item:

```
    xgb_model_1.set_params(**
{'learning_rate':learning_rate})
```

Now that we've set the new learning rate on the model object, we train the model using early stopping as before:

```
    xgb_model_1.fit(X_train, y_train,
eval_set=eval_set,\
```

```
                    eval_metric='auc',\
                    verbose=False,
  early_stopping_rounds=30)
```

After fitting, we obtain the predicted probabilities for the validation set and then use them to calculate the validation ROC AUC. This is added to our list of results using the **append** method:

```
      val_set_pred_proba_2 =
  xgb_model_1.predict_proba(X_val)[:,1]
      val_aucs.append(roc_auc_score(y_val,
  val_set_pred_proba_2))
```

Finally, we also capture the number of rounds required for each learning rate:

```
       best_iters.append(
          int(xgb_model_1.get_booster().\
                          attributes()
  ['best_iteration']))
```

The previous five code snippets should all be run together in one cell. The output should be similar to this:

```
  CPU times: user 1min 23s, sys: 526 ms, total: 1min 24s
  Wall time: 22.2 s
```

Now that we have our results from this hyperparameter search, we can visualize validation set performance and the number of iterations. Since these two metrics are on different scales, we'll want to create a dual *y* axis plot. pandas makes this easy, so first we'll put all the data into a data frame:

```
  learning_rate_df = \
  pd.DataFrame({'Learning rate':learning_rates,\
                'Validation AUC':val_aucs,\
                'Best iteration':best_iters})
```

Now we can visualize performance and the number of iterations for different learning rates like this, noting that:

- We set the index (**`set_index`**) so that the learning rate is plotted on the *x* axis, and the other columns on the *y* axis.
- The **`secondary_y`** keyword argument indicates which column to plot on the right-hand *y* axis.
- The **`style`** argument allows us to specify different line styles for each column plotted. **`-o`** is a solid line with dots, while **`--o`** is a dashed line with dots:

```
mpl.rcParams['figure.dpi'] = 400
learning_rate_df.set_index('Learning rate')\
.plot(secondary_y='Best iteration', style=['-o', '--
o'])
```

The resulting plot should look like this:

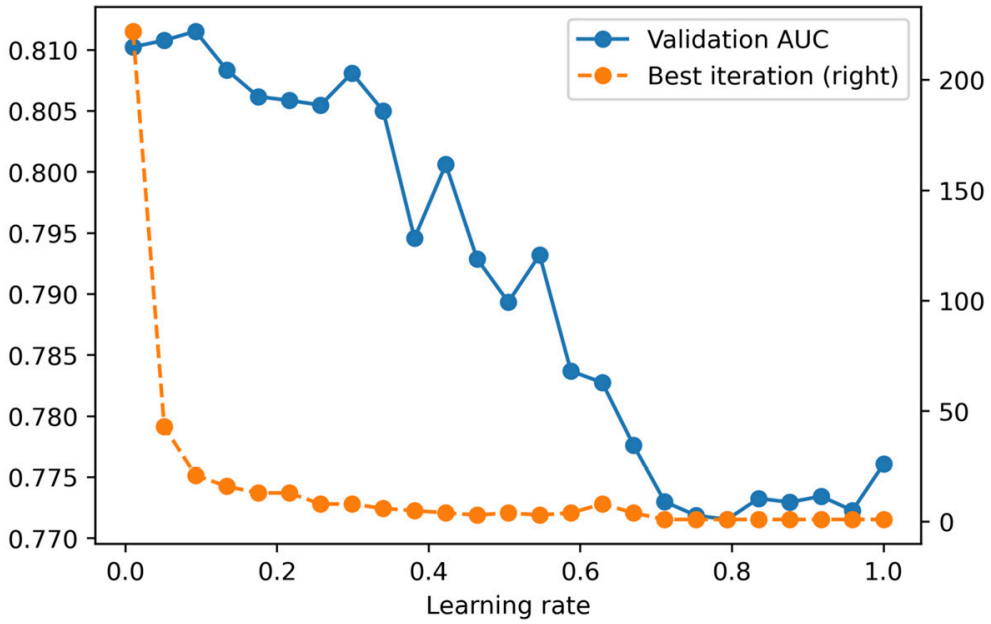

Figure 6.2: XGBoost model performance on a validation set, with the number of boosting rounds until best iteration, for different learning rates

Overall, it appears that smaller learning rates result in better model performance for this synthetic data. By using a learning rate smaller than the default of 0.3, the best performance we can obtain can be seen as follows:

```
max(val_aucs)
```

The output is as follows:

```
0.8115309360232714
```

By adjusting the learning rate, we were able to increase the validation AUC from about 0.80 to 0.81, indicating the benefits of using an appropriate learning rate.

In general, smaller learning rates will usually result in better model performance, although they will require a larger number of boosting rounds, since the contribution of each round is smaller. This will translate into more time required for model training. We can see this in the plot of the number of rounds needed to reach the best iteration in *Figure 6.2*. In this case, it looks like good performance can be attained with fewer than 50 rounds, and the model training time is not that long for this data in any case. For larger datasets, training time may be longer. Depending on how much computational time you have, decreasing the learning rate and training more rounds can be an effective way to increase model performance.

When exploring smaller learning rates, be sure to set the **`n_estimators`** hyperparameter large enough to allow the training process to find the optimal model, ideally in conjunction with early stopping.

Other Important Hyperparameters in XGBoost

We've seen that overfitting in XGBoost can be compensated for by using different learning rates, as well as early stopping. What are some of the other hyperparameters that may be relevant? XGBoost has many hyperparameters and we won't list them all here. You're encouraged to consult the documentation (**https://xgboost.readthedocs.io/en/latest/parameter.html**) for a full list.

In the following exercise, we'll do a grid search over ranges of six hyperparameters, including the learning rate. We will also include **`max_depth`**, which should be familiar from *Chapter 5*, *Decision Trees and Random Forests*, and controls the depth to which trees in the ensemble are grown. Aside from these, we will also consider the following:

- **`gamma`** limits the complexity of trees in the ensemble by only allowing a node to be split if the reduction in the loss function value is greater than a certain amount.
- **`min_child_weight`** also controls the complexity of trees by only splitting nodes if they have at least a certain amount of "sample weight." If all samples have equal weight (as they do for our exercise), this equates to the minimum number of training samples in a node. This is similar to **`min_weight_fraction_leaf`** and **`min_samples_leaf`** for decision trees in scikit-learn.
- **`colsample_bytree`** is a randomly selected fraction of features that will be used to grow each tree in the ensemble. This is similar to the **`max_features`** parameter in scikit-learn (which does the selection at a node level as opposed to the tree level here). XGBoost also makes **`colsample_bylevel`** and **`colsample_bynode`** available to do the feature sampling at each level of each tree, and each node, respectively.
- **`subsample`** controls what fraction of samples from the training data is randomly selected prior to growing a new tree for the ensemble. This is similar to the **`bootstrap`** option for random forests in scikit-learn. Both this and the **`colsample`** parameters limit the information available during model training, increasing the bias of the individual ensemble members, but hopefully also reducing the variance of the overall ensemble and improving out-of-sample model performance.

As you can see, gradient boosted trees in XGBoost implement several concepts that are familiar from decision trees and random forests.

Now, let's explore how these hyperparameters affect model performance.

Exercise 6.01: Randomized Grid Search for Tuning XGBoost Hyperparameters

In this exercise, we'll use a randomized grid search to explore the space of six hyperparameters. A randomized grid search is a good option when you have many values of many hyperparameters you'd like to search over. We'll look at six hyperparameters here. If, for example, there were five values for each of these that we'd like to test, we'd need 5^6 = 15,625 searches. Even if each model fit only took a second, we'd still need several hours to exhaustively search all possible combinations. A randomized grid search can achieve satisfactory results by only searching a random sample of all these combinations. Here, we'll show how to do this using scikit-learn and XGBoost.

The first step in a randomized grid search is to specify the range of values you'd like to sample from, for each hyperparameter. This can be done by either supplying a list of values, or a distribution object to sample from. In the case of discrete hyperparameters such as **`max_depth`**, where there are only a few possible values, it makes sense to specify them as a list. On the other hand, for continuous hyperparameters, such as **`subsample`**, that can vary anywhere on the interval (0, 1], we don't need to specify a list of values. Rather, we can ask that the grid search randomly sample values in a uniform way over this interval. We will use a uniform distribution to sample several of the hyperparameters we consider:

Note

The Jupyter notebook for this exercise can be found at **https://packt.link/TOXso**.

1. Import the **`uniform`** distribution class from **`scipy`** and specify ranges for all hyperparameters to be searched, using a dictionary. **`uniform`** can take two arguments, **`loc`** and **`scale`**, specifying the lower bound of the interval to sample from and the width of the interval, respectively:

```
from scipy.stats import uniform
param_grid = {'max_depth':[2,3,4,5,6,7],
              'gamma':uniform(loc=0.0, scale=3),
              'min_child_weight':list(range(1,151)),
              'colsample_bytree':uniform(loc=0.1,
scale=0.9),
              'subsample':uniform(loc=0.5,
scale=0.5),
              'learning_rate':uniform(loc=0.01,
scale=0.5)}
```

   Here, we've selected parameter ranges based on experimentation and experience. For example with subsample, the XGBoost documentation recommends choosing values of at least 0.5, so we've indicated **`uniform(loc=0.5, scale=0.5)`**, which means sampling from the interval [0.5, 1].

2. Now that we've indicated which distributions to sample from, we need to do the sampling. scikit-learn offers the **`ParameterSampler`** class, which will randomly sample the **`param_grid`** parameters supplied and return as many samples as requested (**`n_iter`**). We also set **`RandomState`** for repeatable results across different runs of the notebook:

```
from sklearn.model_selection import ParameterSampler
rng = np.random.RandomState(0)
n_iter=1000
param_list = list(ParameterSampler(param_grid,
n_iter=n_iter,
                                   random_state=rng))
```

   We have returned the results in a list of dictionaries of specific parameter values, corresponding to locations in the 6-dimensional hy-

perparameter space.

Note that in this exercise, we are iterating through 1,000 hyperparameter combinations, which will likely take over 5 minutes. You may wish to decrease this number for faster results.

3. Examine the first item of **`param_list`**:

```
param_list[0]
```

This should return a combination of six parameter values, from the distributions indicated:

```
{'colsample_bytree': 0.5939321535345923,
'gamma': 2.1455680991172583,
'learning_rate': 0.31138168803582195,
'max_depth': 5,
'min_child_weight': 104,
'subsample': 0.7118273996694524}
```

4. Observe how you can set multiple XGBoost hyperparameters simultaneously with a dictionary, using the **** syntax. First create a new XGBoost classifier object for this exercise.

```
xgb_model_2 = xgb.XGBClassifier(
    n_estimators=1000,
    verbosity=1,
    use_label_encoder=False,
    objective='binary:logistic')
xgb_model_2.set_params(**param_list[0])
```

The output should show the indicated hyperparameters being set:

```
XGBClassifier(base_score=0.5, booster='gbtree',\
              colsample_bylevel=1,
colsample_bynode=1,\
              colsample_bytree=0.5939321535345923,\
              gamma=2.1455680991172583, gpu_id=-1,\
              importance_type='gain',interaction_cons
traints='',\
              learning_rate=0.31138168803582195,\
              max_delta_step=0, max_depth=5,\
              min_child_weight=104, missing=nan,\
```

```
              monotone_constraints='()',
  n_estimators=1000,\
              n_jobs=4, num_parallel_tree=1,\
              random_state=0, reg_alpha=0,
  reg_lambda=1,\
              scale_pos_weight=1,
  subsample=0.7118273996694524,\
              tree_method='exact',
  use_label_encoder=False,\
              validate_parameters=1, verbosity=1)
```

We will use this procedure in a loop to look at all hyperparameter values.

5. The next several steps will be contained in one cell inside a **`for`** loop. First, measure the time it will take to do this, create an empty list to save validation AUCs, and then start a counter:

```
  %%time
  val_aucs = []
  counter = 1
```

6. Open the **`for`** loop, set the hyperparameters, and fit the XGBoost model, similar to the preceding example of tuning the learning rate:

```
  for params in param_list:
      #Set hyperparameters and fit model
      xgb_model_2.set_params(**params)
      xgb_model_2.fit(X_train, y_train,
  eval_set=eval_set,\
                      eval_metric='auc',\
                      verbose=False,
  early_stopping_rounds=30)
```

7. Within the **`for`** loop, get the predicted probability and validation set AUC:

```
      #Get predicted probabilities and save validation
  ROC AUC
```

```
    val_set_pred_proba =
xgb_model_2.predict_proba(X_val)[:,1]
    val_aucs.append(roc_auc_score(y_val,
val_set_pred_proba))
```

8. Since this procedure will take a few minutes, it's nice to print the progress to the Jupyter notebook output. We use the Python remainder syntax, **%**, to print a message every 50 iterations, in other words, when the remainder of **`counter`** divided by 50 equals zero. Finally, we increment the counter:

```
    #Print progress
    if counter % 50 == 0:
        print('Done with {counter} of
{n_iter}'.format(
            counter=counter, n_iter=n_iter))
    counter += 1
```

9. Assembling steps 5-8 in one cell and running the for loop should give output like this:

```
Done with 50 of 1000
Done with 100 of 1000
…
Done with 950 of 1000
Done with 1000 of 1000
CPU times: user 24min 20s, sys: 18.9 s, total: 24min
39s
Wall time: 6min 27s
```

10. Now that we have all the results from our hyperparameter exploration, we need to examine them. We can easily put all the hyperparameter combinations in a data frame, since they are organized as a list of dictionaries. Do this and look at the first few rows:

```
xgb_param_search_df = pd.DataFrame(param_list)
xgb_param_search_df.head()
```

The output should look like this:

	colsample_bytree	gamma	learning_rate	max_depth	min_child_weight	subsample
0	0.593932	2.145568	0.311382	5	104	0.711827
1	0.681305	1.312762	0.455887	2	141	0.691721
2	0.812553	1.586685	0.294022	7	26	0.535518
3	0.178416	0.060655	0.426310	2	83	0.736804
4	0.820820	1.561432	0.349440	2	10	0.768687

Figure 6.3: Hyperparameter combinations from a randomized grid search

11. We can also add the validation set ROC AUCs to the data frame and see what the maximum is:

```
xgb_param_search_df['Validation ROC AUC'] = val_aucs
max_auc = xgb_param_search_df['Validation ROC
AUC'].max()
max_auc
```

The output should be as follows:

```
0.8151220995602575
```

The result of searching over the hyperparameter space is that the validation set AUC is about 0.815. This is larger than the 0.812 we obtained with early stopping and searching over learning rates (*Figure 6.3*), although not much. This means that, for this data, the default hyperparameters (aside from the learning rate) were sufficient to achieve pretty good performance. While we didn't improve performance much with the hyperparameter search, it is instructive to see how the changing values of the hyperparameters affect model performance. We'll examine the marginal distributions of AUCs with respect to each parameter individually in the following steps. This means that we'll look at how the AUCs change as one hyperparameter at a time changes, keeping in mind the fact that the other hyperparameters are also changing in our grid search results.

12. Set up a grid of six subplots for plotting performance against each hyperparameter using the following code, which also adjusts the figure resolution and starts a counter we'll use to loop through the subplots:

```
mpl.rcParams['figure.dpi'] = 400
```

```
fig, axs = plt.subplots(3,2,figsize=(8,6))
counter = 0
```

13. Open a **`for`** loop to iterate through the hyperparameter names, which are the columns of the data frame, not including the last column. Access the axes objects by flattening the 3 x 2 array returned by **`subplot`** and indexing it with **`counter`**. For each hyperparameter, use the **`plot.scatter`** method of the data frame to make a scatter plot on the appropriate axis. The *x* axis will show the hyperparameter, the *y* axis the validation AUC, and the other options help us get black circular markers with white face colors (interiors):

```
for col in xgb_param_search_df.columns[:-1]:
    this_ax = axs.flatten()[counter]
    xgb_param_search_df.plot.scatter(x=col,\
                                     y='Validation
ROC AUC',\
                                     ax=this_ax,
marker='o',\
                                     color='w',\
                                     edgecolor='k',\
                                     linewidth=0.5)
```

14. The data frame's **`plot`** method will automatically create *x* and *y* axis labels. However, since the *y* axis label will be the same for all of these plots, we only need to include it in the first one. So we set all the others to an empty string, `''`, and increment the counter:

```
    if counter > 0:
        this_ax.set_ylabel('')
    counter += 1
```

Since we will be plotting marginal distributions, as we look at how validation AUC changes with a given hyperparameter, all the other hyperparameters are also changing. This means that the relationship may be noisy. To get an idea of the overall trend, we are also going to create line plots with the average value of the validation AUC in each decile of the hyperparameter. Deciles organize data into bins based on whether the values fall into the bottom 10%, the

next 10%, and so on, up to the top 10%. pandas offers a function called **`qcut`**, which cuts a Series into quantiles (a quantile is one of a group of equal-size bins, for example one of the deciles in the case of 10 bins), returning another series of the quantiles, as well as the endpoints of the quantile bins, which you can think of as histogram edges.

15. Use pandas **`qcut`** to generate a series of deciles (10 quantiles) for each hyperparameter (except **`max_depth`**), returning the bin edges (there will be 11 of these for 10 quantiles) and dropping bin edges as needed if there are not enough unique values to divide into 10 quantiles (**`duplicates='drop'`**). Create a list of points halfway between each pair of bin edges for plotting:

```
    if col != 'max_depth':
        out, bins = pd.qcut(xgb_param_search_df[col],
q=10,\
                            retbins=True,
duplicates='drop')
        half_points = [(bins[ix] + bins[ix+1])/2
                       for ix in range(len(bins)-1)]
```

16. For **`max_depth`**, since there are only six unique values, we can use these values directly in a similar way to the deciles:

```
    else:
        out = xgb_param_search_df[col]
        half_points =
np.sort(xgb_param_search_df[col].unique())
```

17. Create a temporary data frame by copying the hyperparameter search data frame, create a new column with the Series of deciles, and use this to find the average value of the validation AUC within each hyperparameter decile:

```
    tmp_df = xgb_param_search_df.copy()
    tmp_df['param_decile'] = out
    mean_df = tmp_df.groupby('param_decile').agg(
        {'Validation ROC AUC':'mean'})
```

18. We can visualize results with a dashed line plot of the decile averages of validation AUC within each grouping, on the same axis as each scatter plot. Close the **`for`** loop and clean up the subplot formatting with **`plt.tight_layout()`**:

```
        this_ax.plot(half_points,\
                     mean_df.values,\
                     color='k',\
                     linestyle='--')
    plt.tight_layout()
```

After running the **`for`** loop, the resulting image should look like this:

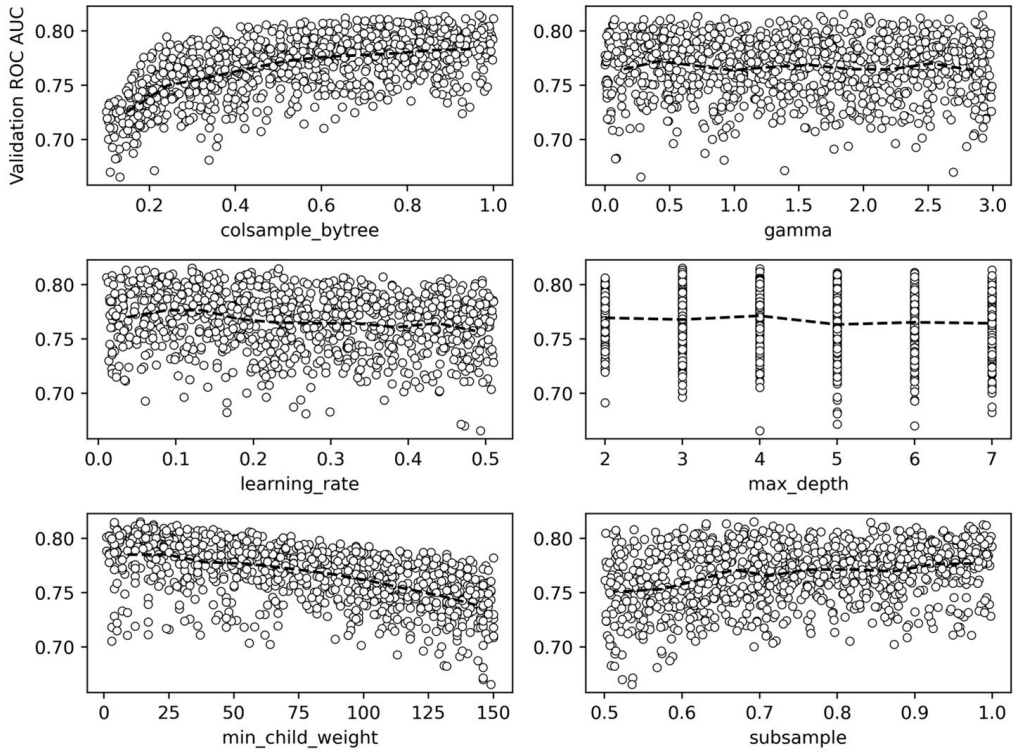

Figure 6.4: Validation AUCs plotted against each hyperparameter, along with the average values within hyperparameter deciles

While we noted that the hyperparameter search in this exercise did not result in a substantial increase in validation AUC over previous efforts in this chapter, the plots in *Figure 6.4* can still show us how XGBoost hyperparameters affect model performance for this particular dataset. One way that XGBoost combats overfitting is by limiting the data available when growing trees, either by randomly selecting only a fraction of the features available to each tree

(**colsample_bytree**), or a fraction of the training samples (**subsample**). However, for this synthetic data, it appears the model performs best when using 100% of the features and samples for each tree; less than this and model performance steadily degrades.
Another way to control overfitting is to limit the complexity of trees in the ensemble, by controlling their **max_depth**, the minimum number of training samples in the leaves (**min_child_weight**), or the minimum reduction in the value of the loss function reduction required to split a node (**gamma**). Neither **max_depth** nor **gamma** appear to have much effect on model performance in our example here, while limiting the number of samples in the leaves appears to be detrimental.
It appears that in this case, the gradient boosting procedure is robust enough on its own to achieve good model performance, without any additional tricks required to reduce overfitting. Similar to what we observed above, however, having a smaller **learning_rate** is beneficial.

19. We can show the optimal hyperparameter combination and the corresponding validation set AUC as follows:

```
max_ix = xgb_param_search_df['Validation ROC AUC'] ==
max_auc
xgb_param_search_df[max_ix]
```

This should return a row of the data frame similar to this:

	colsample_bytree	gamma	learning_rate	max_depth	min_child_weight	subsample	Validation ROC AUC
308	0.83627	1.843039	0.120745	3	14	0.692646	0.815122

Figure 6.5: Optimal hyperparameter combination and validation set AUC

The validation set AUC is similar to what we achieved above (*step 10*) by tuning only the learning rate.

Another Way of Growing Trees: XGBoost's grow_policy

In addition to limiting the maximum depth of trees using a **`max_depth`** hyperparameter, there is another paradigm for controlling tree growth: finding the node where a split would result in the greatest reduction in the loss function, and splitting this node, regardless of how deep it will make the tree. This may result in a tree with one or two very deep branches, while the other branches may not have grown very far. XGBoost offers a hyperparameter called **`grow_policy`**, and setting this to **`lossguide`** results in this kind of tree growth, while the **`depthwise`** option is the default and grows trees to an indicated **`max_depth`**, as we've done in *Chapter 5*, *Decision Trees and Random Forests*, and so far in this chapter. The **`lossguide`** grow policy is a newer option in XGBoost and mimics the behavior of LightGBM, another popular gradient boosting package.

To use the **`lossguide`** policy, it is necessary to set another hyperparameter we haven't discussed yet, **`tree_method`**, which must be set to **`hist`** or **`gpu-hist`**. Without going into too much detail, the **`hist`** method will use a faster way of searching for splits. Instead of looking between every sequential pair of sorted feature values for the training samples in a node, the **`hist`** method builds a histogram, and only considers splits on the edges of the histogram. So, for example, if there are 100 samples in a node, their feature values may be binned into 10 groups, meaning there are only 9 possible splits to consider instead of 99.

We can instantiate an XGBoost model for the **`lossguide`** grow policy as follows, using a learning rate of **`0.1`** based on intuition from our hyperparameter exploration in the previous exercise:

```
xgb_model_3 = xgb.XGBClassifier(
    n_estimators=1000,
```

```
    max_depth=0,
    learning_rate=0.1,
    verbosity=1,
    objective='binary:logistic',
    use_label_encoder=False,
    n_jobs=-1,
    tree_method='hist',
    grow_policy='lossguide')
```

Notice here that we've set **`max_depth=0`**, since this hyperparameter is not relevant for the **`lossguide`** policy. Instead, we are going to set a hyperparameter called **`max_leaves`**, which simply controls the maximum number of leaves in the trees that will be grown. We'll do a hyperparameter search of values ranging from 5 to 100 leaves:

```
max_leaves_values = list(range(5,105,5))
print(max_leaves_values[:5])
print(max_leaves_values[-5:])
```

This should output the following:

```
[5, 10, 15, 20, 25]
[80, 85, 90, 95, 100]
```

Now we are ready to repeatedly fit and validate the model across this range of hyperparameter values, similar to what we've done previously:

```
%%time
val_aucs = []
for max_leaves in max_leaves_values:
    #Set parameter and fit model
    xgb_model_3.set_params(**{'max_leaves':max_leaves})
    xgb_model_3.fit(X_train, y_train,
eval_set=eval_set,\
                    eval_metric='auc', verbose=False,\
                    early_stopping_rounds=30)
    #Get validation score
```

```
    val_set_pred_proba =
xgb_model_3.predict_proba(X_val)[:,1]
    val_aucs.append(roc_auc_score(y_val,
val_set_pred_proba))
```

The output will include the wall time for all of these fits, which was about 24 seconds in testing. Now let's put the results in a data frame:

```
max_leaves_df = \
pd.DataFrame({'Max leaves':max_leaves_values,
              'Validation AUC':val_aucs})
```

We can visualize how the validation AUC changes with the maximum number of leaves, similar to our visualization of the learning rate:

```
mpl.rcParams['figure.dpi'] = 400
max_leaves_df.set_index('Max leaves').plot()
```

This will result in a plot like this:

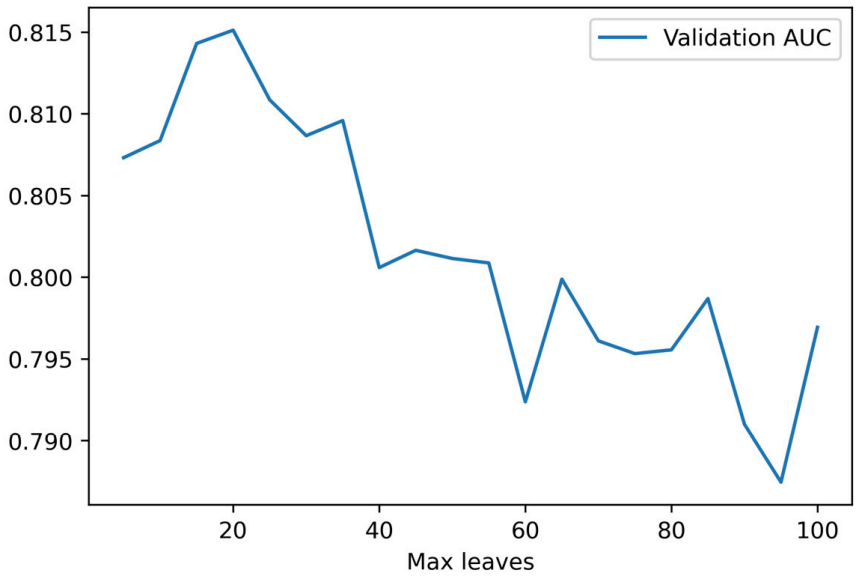

Figure 6.6: Validation AUC against the max_leaves hyperparameter

Smaller values of **`max_leaves`** will limit the complexity of the trees grown for the ensemble, which will ideally increase bias, but also decrease variance for improved out-of-sample performance. We can see this in a higher validation set AUC when the trees are limited to 15 or 20 leaves. What is the maximum validation set AUC?

```
max_auc = max_leaves_df['Validation AUC'].max()
max_auc
```

This should output the following:

```
0.8151200989120475
```

Let's confirm that this maximum validation AUC occurs at **`max_leaves=20`**, as indicated in *Figure 6.6*:

```
max_ix = max_leaves_df['Validation AUC'] == max_auc
max_leaves_df[max_ix]
```

This should return a row of the data frame:

	Max leaves	Validation AUC
3	20	0.81512

Figure 6.7: Optimal max_leaves

By using the **`lossguide`** grow policy, we can achieve performance at least as good as anything else we've tried so far. One key advantage of the **`lossguide`** policy is that, for larger datasets, it can result in training times that are faster than the **`depthwise`** policy, especially for smaller values of **`max_leaves`**. While the dataset here is small enough that this is not of practical importance, this speed may be desirable in other applications.

Explaining Model Predictions with SHAP Values

Along with cutting-edge modeling techniques such as XGBoost, the practice of explaining model predictions has undergone substantial development in recent years. So far, we've learned that logistic regression coefficients, or feature importances from random forests, can provide insight into the reasons for model predictions. A more powerful technique for explaining model predictions was described in a 2017 paper, *A Unified Approach to Interpreting Model Predictions*, by

Scott Lundberg and Su-In Lee (**https://arxiv.org/abs/1705.07874**). This technique is known as **SHAP** (**SHapley Additive exPlanations**) as it is based on earlier work by mathematician Lloyd Shapley. Shapely developed an area of game theory to understand how coalitions of players can contribute to the overall outcome of a game. Recent machine learning research into model explanation leveraged this concept to consider how groups or coalitions of features in a predictive model contribute to the output model prediction. By considering the contribution of different groups of features, the SHAP method can isolate the effect of individual features.

Note

At the time of writing, the SHAP library used in Chapter 6, Gradient Boosting, XGBoost, and SHAP Values, is not compatible with Python 3.9. Hence, if you are using Python 3.9 as your base environment, we suggest that you set up a Python 3.8 environment as described in the Preface.

Some notable aspects of using SHAP values to explain model predictions include:

- SHAP values can be used to make **individualized** explanations of model predictions; in other words, the prediction of a single sample, in terms of the contribution of each feature, can be understood using SHAP. This is in contrast to the feature importance method of explaining random forests that we've already seen, which only considers the average importance of a feature across the model training set.
- SHAP values are calculated relative to a background dataset. By default, this is the training data, although other datasets can be supplied.
- SHAP values are additive, meaning that for the prediction of an individual sample, the SHAP values can be added up to recover the value of the prediction, for example, a predicted probability.

There are different implementations of the SHAP method for various types of models and here we will focus on SHAP for trees (Lundberg et al., 2019, **https://arxiv.org/abs/1802.03888**) to get insights into XGBoost model predictions on our validation set of synthetic data. First, let's refit **xgb_model_3** from the previous section with the optimal number of **max_leaves**, **20**:

```
%%time
xgb_model_3.set_params(**{'max_leaves':20})
xgb_model_3.fit(X_train, y_train,\
                eval_set=eval_set,\
                eval_metric='auc',
                verbose=False,\
                early_stopping_rounds=30)
```

Now we're ready to start calculating SHAP values for the validation dataset. There are 40 features and 1,000 samples here:

```
X_val.shape
```

This should output the following:

```
(1000, 40)
```

To automatically label the plots we can make with the **shap** package, we'll put the validation set features in a data frame with column names. We'll use a list comprehension to make generic feature names, for example, "Feature 0, Feature 1, ..." and create the data frame as follows:

```
feature_names = ['Feature
{number}'.format(number=number)
                 for number in range(X_val.shape[1])]
X_val_df = pd.DataFrame(data=X_val,
columns=feature_names)
X_val_df.head()
```

The **dataframe** head should look like this:

	Feature 0	Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Feature 6	Feature 7	Feature 8	Feature 9	...
0	1.852885	-2.170293	1.057288	0.441873	-0.803131	-0.025139	-0.037143	0.037565	1.163995	0.678410	...
1	-0.818316	-1.126948	0.647810	0.092433	-1.030356	0.754323	-0.351566	-0.523476	1.144878	0.219172	...
2	0.020271	-0.758004	-1.136195	0.473366	1.291465	0.890423	-2.217706	-2.030749	1.768624	-2.106202	...
3	-0.271543	-0.366639	-1.139614	-0.753586	1.427853	1.249856	0.060528	-0.374193	0.047770	0.640638	...
4	-0.549078	0.494648	-1.266778	-0.292728	1.459779	0.497898	-0.618724	-1.225373	0.171507	0.833027	...

Figure 6.8: Data frame of the validation features

With the trained model, **xgb_model_3**, and the data frame of validation features, we're ready to create an **explainer** interface. The SHAP package has various kinds of explainers and we'll use the one specifically for tree models:

```
explainer = shap.explainers.Tree(xgb_model_3,
data=X_val_df)
```

This has created an explainer using the model validation data as the background dataset. Now we are ready to use the explainer to obtain SHAP values. The SHAP package makes this very simple. All we need to do is pass in the dataset we want explanations for:

```
shap_values = explainer(X_val_df)
```

That's all there is to it! What is this variable, **shap_values**, that has been created? If you examine the contents of the **shap_values** variable directly, you will see that it contains three attributes. The first is **values**, which contains the SHAP values. Let's examine the shape:

```
shap_values.values.shape
```

This should return the following:

```
(1000, 40)
```

Because SHAPs provide individualized explanations, there is a row for each of the 1,000 samples in the validation set. There are 40 columns because we have 40 features and SHAP values tell us the contribution of each feature to the prediction for each sample. **shap_values** also contains a **base_values** attribute, which is the naïve prediction before any feature contributions are considered, also defined as the average

prediction across the entire dataset. There is one of these for each sample (1,000). Finally, there is also a **`data`** attribute, which contains the feature values. All of this information can be combined in various ways to explain model predictions.

Thankfully, not only does the **`shap`** package provide fast and convenient methods for calculating SHAP values, but it also provides a rich suite of visualization techniques. One of the most popular is a SHAP summary plot, which visualizes the contribution of each feature to each sample. Let's create this plot and then understand what is being shown. Please note that most interesting SHAP visualizations use color, so if you're reading in black and white, please refer to the GitHub repository for color figures:

```
mpl.rcParams['figure.dpi'] = 75
shap.summary_plot(shap_values.values, X_val_df)
```

This should produce the following:

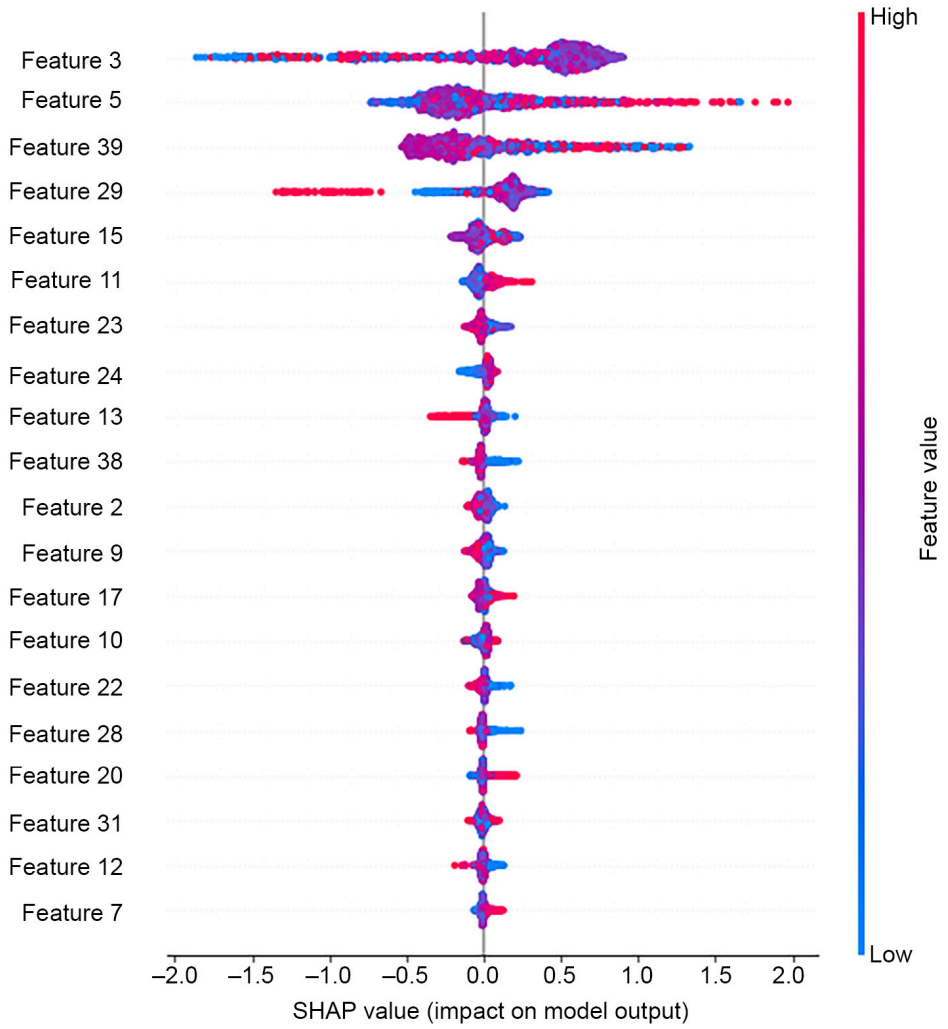

Figure 6.9: SHAP summary plot for the synthetic data validation set

Note

If you're reading the print version of this book, you can download and browse the color versions of some of the images in this chapter by visiting the following link: **https://packt.link/ZFiYH**

Figure 6.9 contains a lot of information to help us explain the model. The summary plot may contain up to 40,000 plotted points, one for each of the 40 features and each of the 1,000 validation samples (although only the first 20 features are shown by default). Let's start by understanding the *x* axis. The SHAP value indicates the additive contribution of each feature value to the prediction for a sample. SHAP values are shown here relative to the expected values, which are the **`base_values`** described earlier. So if a given feature has a small impact

on the prediction for a given sample, it will not tend to move the prediction very far from the expected value, and the SHAP value will be close to zero. However if a feature has a large effect, which, in the case of our binary classification problem, means that the predicted probability will be pushed closer to 0 or 1, the SHAP value will be further from 0. Negative SHAP values indicate a feature moving the prediction closer to 0, and positive SHAP values indicate closer to 1.

Note that the SHAP values shown in *Figure 6.9* cannot be directly interpreted as predicted probabilities. By default, SHAP values for the XGBoost binary classification model with the **`binary:logistic`** objective function are calculated and plotted using the log-odds representation of probability, which was introduced in *Chapter 3*, *Details of Logistic Regression and Feature Exploration* in the *Why Is Logistic Regression Considered a Linear Model?* section. This means that they can be added and subtracted, or in other words, we can perform linear transformations on them.

What about the color of the dots in *Figure 6.9*? These represent the values of the features for each sample, with red meaning a higher value and blue lower. So, for example, we can see in the fourth row of the plot that the lowest SHAP values come from high feature values (red dots) for Feature 29.

The vertical arrangement of the dots, in other words, the width of the band of dots for each feature, indicates how many dots there are at that location on the *x* axis. If there are many samples, the band of dots will be wider.

The vertical arrangement of features in the diagram is based on feature importance. The most important features, in other words, those with the largest average effect (mean absolute SHAP value) on model predictions, are placed at the top of the list.

While the summary plot in *Figure 6.9* is a great way to look at all of the most important features and their SHAP values at once, it may not reveal some interesting relationships. For example, the most important feature, Feature 3, appears to have a large clump of purple dots (middle of the range of feature values) that have positive SHAP values, while the negative SHAP values for this feature may result from high or low feature values.

What is going on here? Often, when the effects of features seem unclear from a SHAP summary plot, the tree-based model we are using is capturing interaction effects between features. To gain additional insight into individual features and their interactions with others, we can use a SHAP scatter plot. Firstly, let's make a simple scatter plot of the SHAP values of Feature 3. Note that we can index the **`shap_values`** object in a similar way to a data frame:

```
shap.plots.scatter(shap_values[:,'Feature 3'])
```

This should produce the following plot:

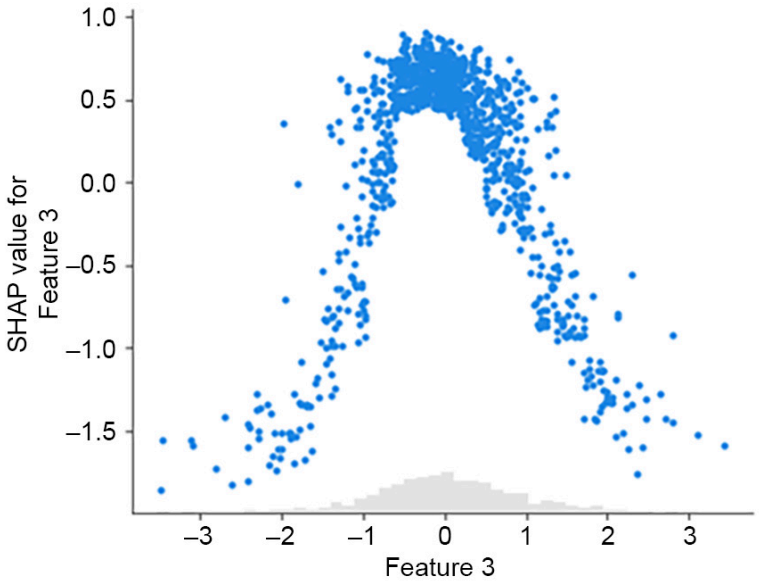

Figure 6.10: Scatter plot of SHAP values for Feature 3

From *Figure 6.10*, we can tell pretty much the same information that we could from the summary plot of *Figure 6.9*: feature values in the middle of the range have high SHAP values, while those at the ex-

tremes are lower. However, the **`scatter`** method also allows us to color the points of the scatter plot by another feature value, so we can see whether there are interactions between the features. We'll color points by the second most important feature, Feature 5:

```
shap.plots.scatter(shap_values[:,'Feature 3'],
                   color=shap_values[:,'Feature 5'])
```

The resulting plot should look like this:

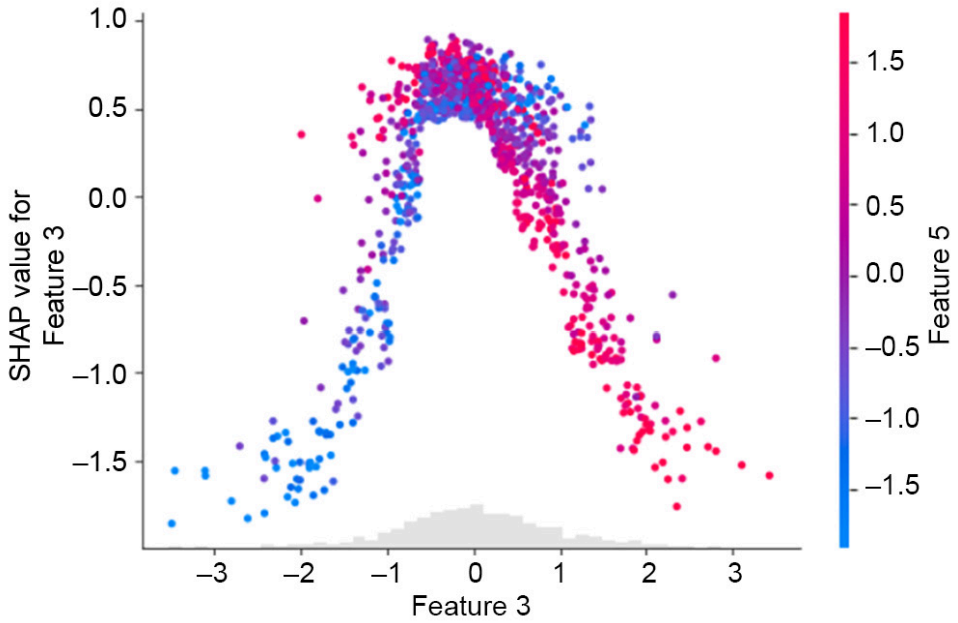

Figure 6.11: Scatter plot of SHAP values for Feature 3, colored by feature values of Feature 5. Arrows A and B indicated interesting interaction effects between these features

Figure 6.11 shows an interesting interaction between Feature 3 and Feature 5. When samples are in the middle of the range of feature values for Feature 3, in other words, at the top of the hump shape in *Figure 6.11*, the color of dots appears to get more red going from the bottom to the top of the cluster of dots here (arrow A). This means that for feature values in the middle of the Feature 3 range, as the value of Feature 5 increases, so does the SHAP value for Feature 3. We can also see that as feature values of Feature 3 increase along the *x* axis from the middle toward the top of the range, this relationship reverses to where higher feature values for Feature 5 begin to correspond to

lower SHAP values for Feature 3 (arrow B). So the interaction with Feature 5 appears to have a substantial impact on the SHAP values for Feature 3.

The complex relationships depicted in *Figure 6.11* show how increasing a feature value may lead to either increasing or decreasing SHAP values when interaction effects are present. The specific reasons for the patterns in *Figure 6.11* relate to the creation of the synthetic dataset we are modeling, where we specified multiple clusters in the feature space. As discussed in *Chapter 5*, *Decision Trees and Random Forests*, in the *Using Decision Trees: Advantages and Predicted Probabilities* section, tree-based models such as XGBoost are able to effectively model clusters of points in multi-dimensional feature space that belong to a certain class. SHAP explanations can help us to understand how the model is making these representations.

Here, we've used synthetic data, and the features have no real-world interpretation, so we can't assign any meaning to interactions we observe. However, with real-world data, detailed exploration with SHAP values and interactions can provide insight into how a model is representing complex relationships between attributes of customers or users, for example. SHAP values are also useful since they can provide explanations relative to any background dataset. While logistic regression coefficients and feature importances of random forests are determined entirely by the model training data, SHAP values can be calculated for any background dataset; so far in this chapter, we've been using the validation data. This provides an opportunity, when predicted models are deployed in a production environment, to understand how new predictions are being made. If the SHAP values for new predictions are very different from those of model training and test data, this may indicate that the nature of incoming data has changed, and it may be time to consider developing a new model. We'll consider these practical aspects of using models in the real world in the final chapter.

Exercise 6.02: Plotting SHAP Interactions, Feature Importance, and Reconstructing Predicted Probabilities from SHAP Values

In this exercise, you'll become more familiar with using SHAP values to provide visibility into the workings of a model. First, we'll take an alternate look at the interaction between Features 3 and 5, and then use SHAP values to calculate feature importances similar to what we did with a random forest model in *Chapter 5, Decision Trees and Random Forests*. Finally, we'll see how model outputs can be obtained from SHAP values, taking advantage of their additive property:

Note

The Jupyter notebook for this exercise can be found at **https://packt.link/JcMoA**.

1. Given the preliminary steps accomplished in this section already, we can take another look at the interaction between Features 3 and 5, the two most important features of the synthetic dataset. Use the following code to make an alternate version of *Figure 6.11*, except this time, look at the SHAP values of Feature 5, colored by those of Feature 3:

```
shap.plots.scatter(shap_values[:,'Feature 5'],
                   color=shap_values[:,'Feature 3'])
```

The resulting plot should look like this:

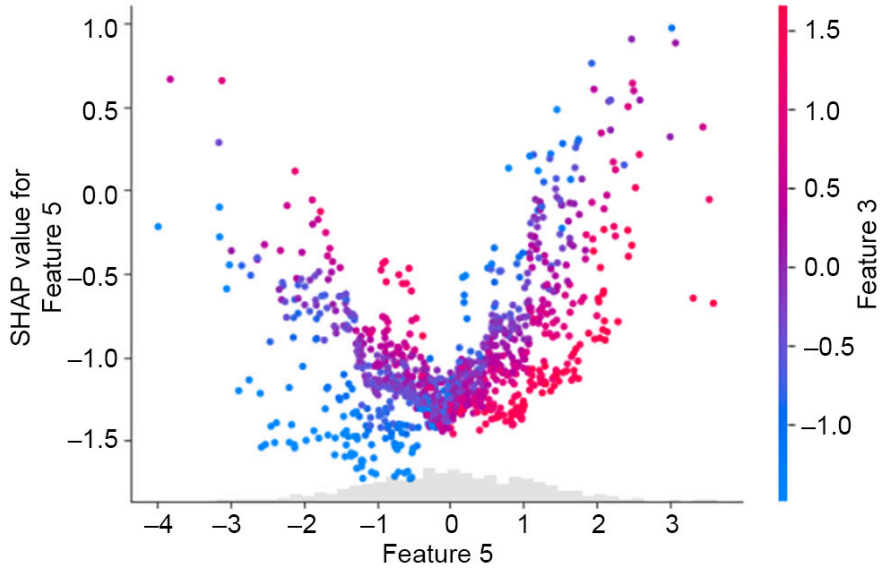

Figure 6.12: Scatter plot of SHAP values for Feature 5, colored by feature values of Feature 3

As opposed to *Figure 6.11*, here we are seeing the SHAP values of Feature 5. In general, from the scatter plot, we can see that SHAP values tend to increase as feature values increase for Feature 5. However there are certainly counterexamples to that general trend, as well as an interesting interaction with Feature 3: for a given value of Feature 5, which can be thought of as a vertical slice from the image, the color of the dots can either become more red, going from the bottom to the top, for negative feature values, or less red for positive feature values. This means that for a given value of Feature 5, its SHAP value depends on the value of Feature 3. This is a further illustration of the interesting interaction between Features 3 and 5. In a real project, which plot you would choose to show depends on what kind of story you want to tell with the data, relating to what real-world quantities Features 3 and 5 might represent.

2. Create a feature importance bar plot using the following code:

```
mpl.rcParams['figure.dpi'] = 75
shap.summary_plot(shap_values.values, X_val,
plot_type='bar')
```

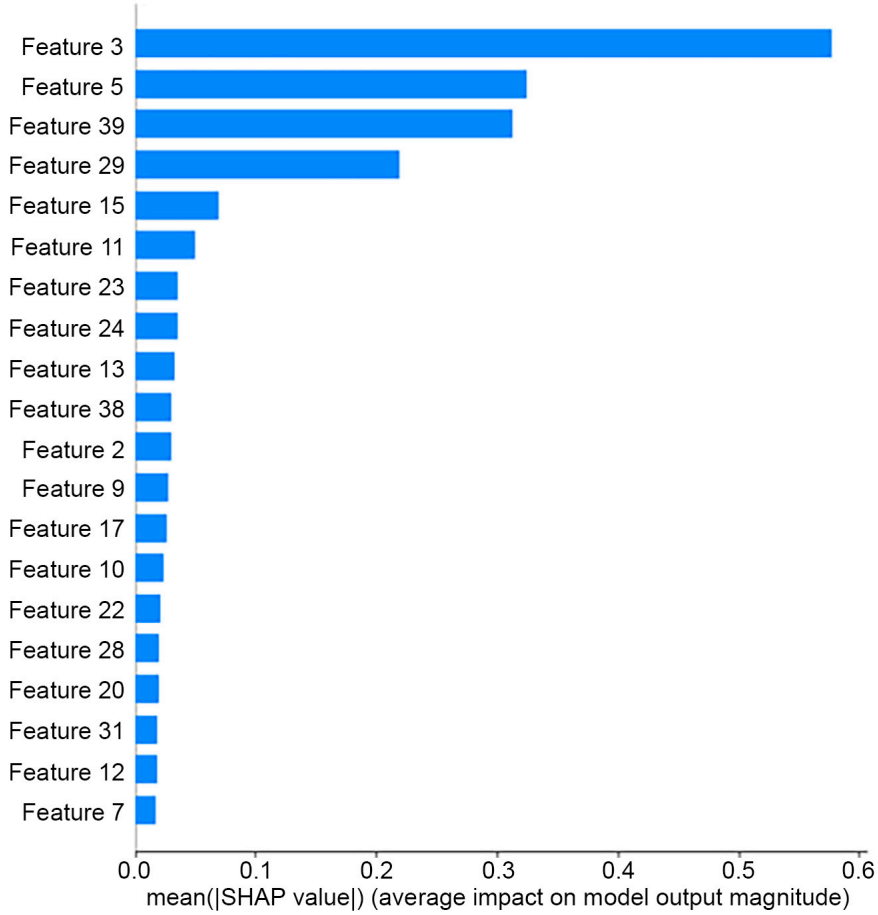

Figure 6.13: Feature importance bar plot using SHAP values

The feature importance bar plot gives a visual presentation of information similar to that obtained in *Exercise 5.03, Fitting a Random Forest,* in *Chapter 5, Decision Trees and Random Forests*, with a random forest: this is a single number for each feature, representing how important it is overall for a dataset.

Do these results make sense? Recall that we created this synthetic data with three informative features and two redundant ones. In *Figure 6.13*, it appears that there are four features that are substantially more important than all the others, so perhaps one of the redundant features was created in such a way that XGBoost selected it for splitting nodes fairly often, but the other redundant feature was not used as much.

Compared to the feature importances we found in *Chapter 5*, *Decision Trees and Random Forests*, the ones here are a bit different. The feature importances we can obtain from scikit-learn for a random forest model are calculated using the decrease in node impurity due to the feature as well as the fraction of training samples

split by the feature. By contrast, feature importances using SHAP values are calculated as follows: first, the absolute value of all the SHAP values (**`shap_values.values`**) is taken, then an average of all the samples is taken for each feature, as implied by the *x* axis label. The interested reader can confirm this by calculating these metrics directly from **`shap_values`**.

Now that we've familiarized ourselves with a range of uses of SHAP values, let's see how their additive property allows the reconstruction of predicted probabilities.

3. SHAP values are calculated relative to the expected value, or base value, of a model. This can be interpreted as the average prediction over all samples in the background dataset. However, the prediction will be in units of log-odds as opposed to probability, as mentioned earlier, to support additivity. The expected value of a model can be accessed from the explainer object as follows:

```
explainer.expected_value
```

The output should look like this:

```
-0.30949621941894295
```

This information isn't particularly useful on its own. However, it gives us the baseline from which we can reconstruct predicted probabilities.

4. Recall that the shape of the SHAP values matrix is the number of samples by the number of features. In our exercise with the validation data, here that would be 1,000 by 40. To add up all the SHAP values for each sample, we therefore want to take a sum over the column axis (**`axis=1`**). This adds all the feature contributions, effectively providing the offset from the expected value. If we add the expected value to this, we then have the following predictions:

```
shap_sum = shap_values.values.sum(axis=1) +
explainer.expected_value
shap_sum.shape
```

This should return the following:

```
(1000,)
```

Indicating we now have a single number for each sample.

However, these predictions are in log-odds space. To transform them to probability space, we need to apply the logistic function introduced in *Chapter 3, Details of Logistic Regression and Feature Exploration*.

5. Apply the logistic transformation to log-odds predictions like this:

```
shap_sum_prob = 1 / (1 + np.exp(-1 * shap_sum))
```

Now we'd like to compare the predicted probabilities obtained from SHAP values with direct model output for confirmation.

6. Obtain predicted probabilities for the model validation set and check the shape with this code:

```
y_pred_proba = xgb_model_3.predict_proba(X_val)[:,1]
y_pred_proba.shape
```

The output should be as follows:

```
(1000,)
```

This is the same shape as our SHAP-derived predictions, as expected.

7. Put the model output and sums of SHAP values together in a data frame for side-by-side comparison, and spot check a random selection of five rows:

```
df_check = pd.DataFrame(
    {'SHAP sum':shap_sum_prob,
     'Predicted probability':y_pred_proba})
df_check.sample(5, random_state=1)
```

The output should confirm that the two methods have identical results:

	SHAP sum	Predicted probability
507	0.497260	0.497260
818	0.466160	0.466160
452	0.881343	0.881343
368	0.145347	0.145347
242	0.481065	0.481065

Figure 6.14: Comparison of SHAP-derived predicted probabilities and those obtained directly from XGBoost

The spot check indicates that these five samples have identical values. While the values may not be precisely equal due to rounding errors of machine arithmetic, you could use NumPy's **`allclose`** function to ensure they're the same within a user-configurable amount of rounding error.

8. Ensure that the SHAP-derived probabilities and model output probabilities are all very close to each other like this:

```
np.allclose(df_check['SHAP sum'],\
            df_check['Predicted probability'])
```

The output should be as follows:

```
True
```

This indicates that all elements of both columns are equal within rounding error. **`allclose`** is useful for when rounding errors are present and exact equality (testable with **`np.array_equal`**) would not hold.

By now, you should be getting an impression of the power of SHAP values to help understand machine learning models. The sample-specific, individualized nature of SHAP values opens up the possibility of very detailed analyses, which could help answer a wide variety of potential questions from business stakeholders such as "How would the model make predictions for people like this?" or "Why did the model make this prediction for this specific person"? Now that we're familiar with XGBoost and SHAP values, two state-of-the-art machine learning techniques, we return to the case study data to apply them.

Missing Data

As a final note on the use of both XGBoost and SHAP, one valuable trait of both packages is their ability to handle missing values. Recall that in *Chapter 1*, *Data Exploration and Cleaning*, we found that some samples in the case study data had missing values for the **`PAY_1`** feature. So far, our approach has been to simply remove these samples from the dataset when building models. This is because, without specifically ad-

dressing the missing values in some way, the machine learning models implemented by scikit-learn cannot work with the data. Ignoring them is one approach, although this may not be satisfactory as it involves throwing data away. If it's a very small fraction of the data, this may be fine; however, in general, it's good to be able to know how to deal with missing values.

There are several approaches for imputing missing values of features, such as filling them in with the mean or mode of the non-missing values of that feature, or a randomly selected value from the non-missing values. You can also build a model outputting the feature in question as the response variable, with all the other features acting as features for this new model, and then predict the missing feature values. These approaches were explored in the first edition of this book (**https://packt.link/oLb6C**). However, since XGBoost typically performs at least as well as other machine learning models for binary classification tasks with tabular data like we're using here, and handles missing values, we'll forego more in-depth exploration of imputing missing values and let XGBoost do the work for us.

How does XGBoost handle missing data? At every opportunity to split a node, XGBoost considers only the non-missing feature values. If a feature with missing values is chosen to make a split, the samples with missing values for that feature are then sent down the optimal path to one of the child nodes, in terms of minimizing the loss function.

Saving Python Variables to a File

In the activity for this chapter, to write to and read from files we'll use a new python statement (**`with`**) and the **`pickle`** package. **`with`** statements make it easier to work with files since they both open and close the file, instead of the user needing to do this separately. You can use code snippets like this to save variables to a file:

```
with open('filename.pkl', 'wb') as f:
```

```
    pickle.dump([var_1, var_2], f)
```

where **filename.pkl** is your chosen file path, **'wb'** indicates the file is open for writing in a binary format, and **pickle.dump** saves a list of variables **var_1** and **var_2** to the file. To open this file and load these variables, possibly into a separate Jupyter Notebook, the code is similar but now the file needs to be opened for reading in a binary format (**'rb'**):

```
with open('filename.pkl', 'rb') as f:
    var_1, var_2 = pickle.load(f)
```

Activity 6.01: Modeling the Case Study Data with XGBoost and Explaining the Model with SHAP

In this activity, we'll take what we've learned in this chapter with a synthetic dataset and apply it to the case study data. We'll see how an XGBoost model performs on a validation set and explain the model predictions using SHAP values. We have prepared the dataset for this activity by replacing the samples that had missing values for the **PAY_1** feature, that we had previously ignored, while maintaining the same train/test split for the samples with no missing values. You can see how the data was prepared in the Appendix to the notebook for this activity.

Note

The Jupyter notebook containing the solution as well as the appendix can be found here: **https://packt.link/YFb4r**.

1. Load the case study data that has been prepared for this exercise. The file path is **../../Data/Activity_6_01_data.pkl** and the variables are: **features_response, X_train_all, y_train_all, X_test_all, y_test_all**.
2. Define a validation set to train XGBoost with early stopping.
3. Instantiate an XGBoost model. Use the **lossguide** grow policy to enable the examination of validation set performance for several val-

ues of **`max_leaves`**.

4. Create a list of values of **`max_leaves`** from 5 to 200, counting by 5's.
5. Create the evaluation set for early stopping.
6. Loop through hyperparameter values and create a list of validation ROC AUCs, using the same technique as in *Exercise 6.01: Randomized Grid Search for Tuning XGBoost Hyperparameters*.
7. Create a data frame of the hyperparameter search results and plot the validation AUC against **`max_leaves`**.
8. Observe the number of **`max_leaves`** corresponding to the highest ROC AUC on the validation set.
9. Refit the XGBoost model with the optimal hyperparameter. So that we can examine SHAP values for the validation set, make a data frame of this data.
10. Create a SHAP explainer for our new model using the validation data as the background dataset, obtain the SHAP values, and make a summary plot.
11. Make a scatter plot of **`LIMIT_BAL`** SHAP values, colored by the feature with the strongest interaction.
12. Save the trained model along with the training and test data to a file.

    Note

    The solution to this activity can be found via **this link**.

Summary

In this chapter, we've learned some of the most cutting-edge techniques for building machine learning models with tabular data. While other types of data, such as image or text data, warrant exploration with different types of models such as neural networks, many standard business applications leverage tabular data. XGBoost and SHAP are some of the most advanced and popular tools you can use to build and understand models with this kind of data. Having gained familiarity and practical experience using these tools with synthetic data, in the following activity, we return to the dataset for the case study and

see how we can use XGBoost to model it, including the samples with missing feature values, and use SHAP values to understand the model.